



MATLAB赋能《信号与系统》沉浸式教学—— 基于MCP+WorkBuddy的智能体尝试

汇报人：胡浩基

浙江大学信息与电子工程学院

MATLAB Educator Workshop 2026/05/14



MATLAB MCP + 腾讯WorkBuddy的智能体尝试

(1) 安装MATLAB，安装腾讯WorkBuddy。

CodeBuddy WorkBuddy 公测 [定价](#) [文档](#) [博客](#) [API文档](#) [中国站](#) [登录](#) [下载](#)

Work Smart , Not Hard

腾讯版 AI 办公助手

WorkBuddy 是 AI 原生的桌面智能体工作台，可在手机主流 IM 下指令，AI 自动干活交付
[免部署](#) · [安装即用](#) | [预集成](#) · 模型一站搞定，兼容龙虾 Skills | [高安全](#) · 腾讯网关护航

 [下载 WorkBuddy](#) 

[Claw 快捷接入指南](#)



MATLAB MCP + 腾讯WorkBuddy的智能体尝试

(2) 在腾讯WorkBuddy中安装MATLAB MCP。

WorkBuddy 编辑(E) 窗口(W) 帮助(H)

WorkBuddy v4.10.4

搜索任务

+

+ 新建任务

🔗 Claw

👤 专家

🔧 技能

🕒 自动化

任务

⚠️ 我想在WorkBuddy里配置MA... 1天前

✓ 在D:\matlab\signals and sys... 11天前

胡浩基

>

WorkBuddy

我想在WorkBuddy里配置MATLAB的MCP，步骤在这

我想在WorkBuddy里配置MATLAB的MCP，步骤在这里：<https://github.com/matlab/matlab-mcp-core-server>。我已经下载过了，能帮我配置好吗？

WorkBuddy

> 4 个工具调用, 3 条过程消息

一切就绪，配置已经是完整正确的状态。来汇报一下情况：

✅ MATLAB MCP 已配置完毕，无需任何更改

你的 `mcp.json` 已经有一份完整且正确的配置：

json

```
{
  "mcpServers": {
    "matlab": {
      "type": "stdio",
      "command": "C:\\Users\\Lenovo\\.workbuddy\\matlab-mcp-core-server-win64.exe",
      "args": [
        "--matlab-root=C:\\Program Files\\MATLAB\\R2025b"
      ]
    }
  }
}
```

@

🔗

输入消息...

Craft

Auto

Skills

🔊

🔔

内容由 AI 生成，请核实重要信息。

MATLAB MCP + 腾讯WorkBuddy的智能体尝试

(2) 在腾讯WorkBuddy中安装MATLAB MCP。

WorkBuddy 编辑(E) 窗口(W) 帮助(H)

WorkBuddy v4.10.4

🔍 搜索任务

+

+ 新建任务

🔗 Claw

👤 专家

🔧 技能

🕒 自动化

任务

⚠️ 我想在WorkBuddy里配置MA... 1天前

✓ 在D:\matlab\signals and sys... 11天前

胡浩基

>

WorkBuddy

我想在WorkBuddy里配置MATLAB的MCP，步骤在这

我想在WorkBuddy里配置MATLAB的MCP，步骤在这里：<https://github.com/matlab/matlab-mcp-core-server>。我已经下载过了，能帮我配置好吗？

WorkBuddy

> 4 个工具调用, 3 条过程消息

一切就绪，配置已经是完整正确的状态。来汇报一下情况：

✅ MATLAB MCP 已配置完毕，无需任何更改

你的 `mcp.json` 已经有一份完整且正确的配置：

json

```
{
  "mcpServers": {
    "matlab": {
      "type": "stdio",
      "command": "C:\\Users\\Lenovo\\.workbuddy\\matlab-mcp-core-server-win64.exe",
      "args": [
        "--matlab-root=C:\\Program Files\\MATLAB\\R2025b"
      ]
    }
  }
}
```

@

🔗

输入消息...

Craft

Auto

Skills

🔊

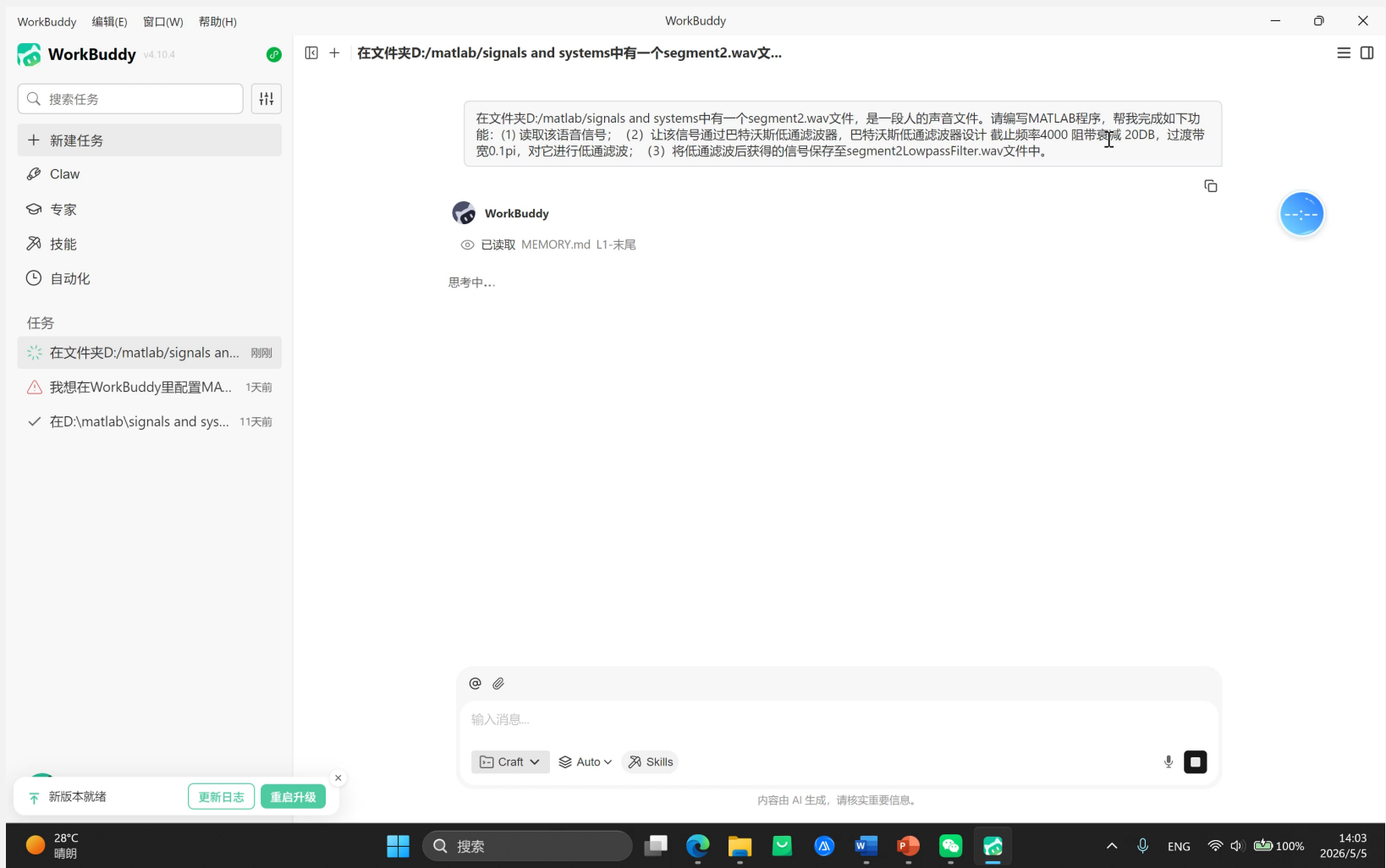
🔔

内容由 AI 生成，请核实重要信息。

MATLAB MCP + 腾讯WorkBuddy的智能体尝试

示例1：信号经过低通滤波器

提示词：在文件夹D:/matlab/signals and systems中有一个segment2.wav文件，是一段人的声音文件。请编写MATLAB程序，帮我完成如下功能：（1）读取该语音信号；（2）让该信号通过巴特沃斯低通滤波器，巴特沃斯低通滤波器设计 截止频率4000 阻带衰减 20DB，过渡带宽0.1pi，对它进行低通滤波；（3）将低通滤波后获得的信号保存至segment2LowpassFilter.wav文件中。



MATLAB MCP + 腾讯WorkBuddy的智能体尝试

示例1：信号经过低通滤波器

```
%% 巴特沃斯低通滤波器设计与应用
% 功能：读取语音信号，设计巴特沃斯低通滤波器，滤波后保存
% 参数：截止频率 4000 Hz，阻带衰减 20 dB，过渡带宽 0.1 $\pi$ 

clc; clear; close all;

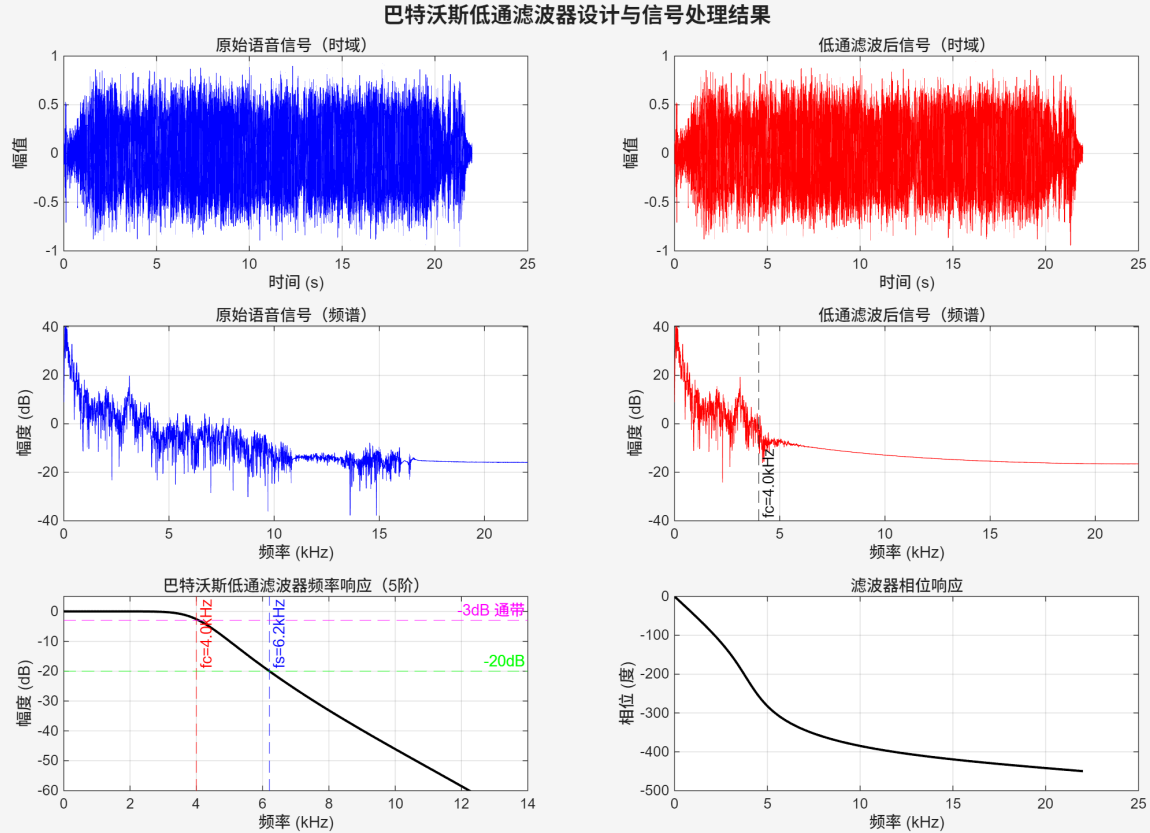
%% (1) 读取语音信号
inputFile = 'D:/matlab/signals and systems/segment2.wav';
outputFile = 'D:/matlab/signals and systems/segment2LowpassFilter.wav';

[x, fs] = audioread(inputFile);
fprintf('=== 读取原始语音信号 ===\n');
fprintf('  采样率      : %d Hz\n', fs);
fprintf('  采样点数     : %d\n', length(x));
fprintf('  声道数       : %d\n', size(x, 2));
fprintf('  时长         : %.2f 秒\n', length(x)/fs);

%% (2) 设计巴特沃斯低通滤波器
% 参数说明：
%   截止频率 fc = 4000 Hz
%   阻带衰减 Rs = 20 dB
```



原始音频

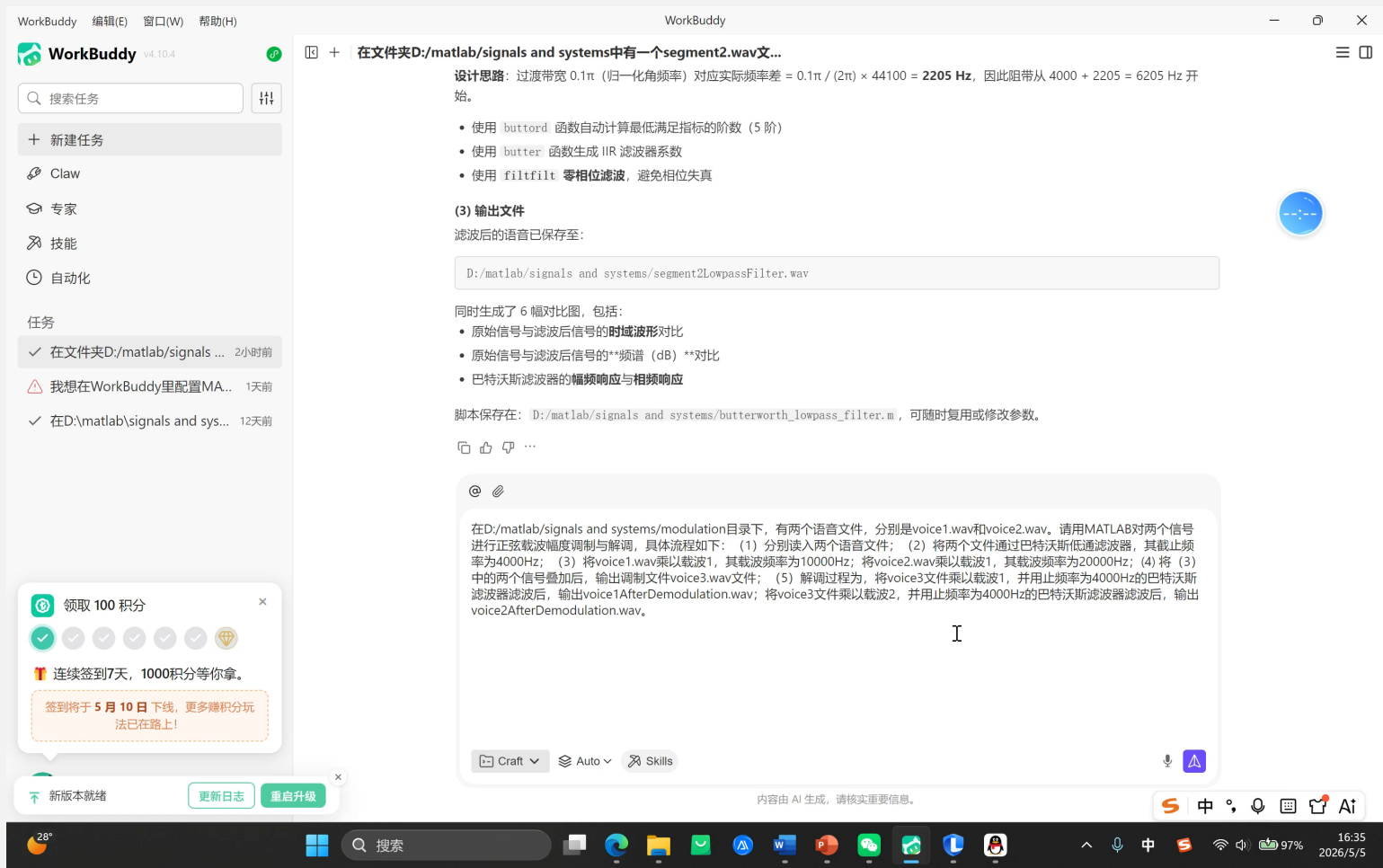


低通滤波后的音频

MATLAB MCP + 腾讯WorkBuddy的智能体尝试

示例2：信号的调制与解调

提示词:在D:/matlab/signals and systems/modulation目录下，有两个语音文件，分别是voice1.wav和voice2.wav。请用MATLAB对两个信号进行正弦载波幅度调制与解调，具体流程如下：（1）分别读入两个语音文件；（2）将两个文件通过巴特沃斯低通滤波器，其截止频率为4000Hz；（3）将voice1.wav乘以载波1，其载波频率为10000Hz；将voice2.wav乘以载波1，其载波频率为20000Hz；（4）将（3）中的两个信号叠加后，输出调制文件 voice3.wav 文件；（5）解调过程为，将 voice3 文件乘以载波1，并用截止频率为4000Hz的巴特沃斯滤波器滤波后，输出 voice1AfterDemodulation.wav；将voice3文件乘以载波2，并用截止频率为4000Hz的巴特沃斯滤波器滤波后，输出voice2AfterDemodulation.wav。



MATLAB MCP + 腾讯WorkBuddy的智能体尝试

示例2：信号的调制与解调

%% 频分复用（FDM）调制与解调

% 功能：

% 1. 读入 voice1.wav / voice2.wav

% 2. 巴特沃斯低通滤波（截止 4000 Hz）

% 3. 幅度调制：voice1 $\times \cos(2\pi \times 10000 \times t)$, voice2 $\times \cos(2\pi \times 20000 \times t)$

% 4. 叠加后保存为 voice3.wav

% 5. 解调：voice3 $\times$ 对应载波 $\rightarrow$ 低通滤波 $\rightarrow$ 保存 voice1/2AfterDemodulation.wav

%

% 参数设定：

% 载波1频率 fc1 = 10000 Hz

% 载波2频率 fc2 = 20000 Hz

% 低通截止频率 flp = 4000 Hz（调制前滤波 + 解调后滤波均使用）

clc; clear; close all;

%% ===== 路径配置 =====

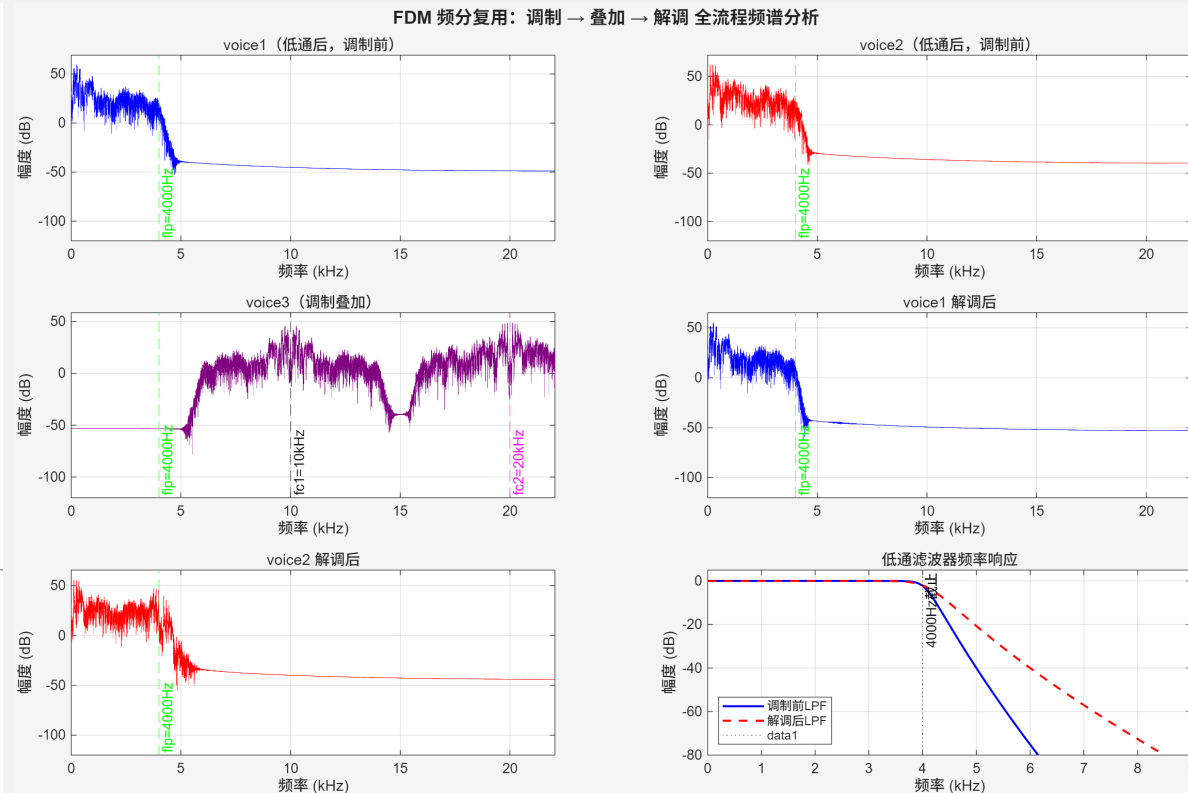
workDir = 'D:/matlab/signals and systems/modulation/';

f_v1 = [workDir 'voice1.wav'];

f_v2 = [workDir 'voice2.wav'];

f_v3 = [workDir 'voice3.wav'];

f_dem1 = [workDir 'voice1AfterDemodulation.wav'];



Voice1.wav



调制后的声音文件Voice3.wav



Voice2.wav



Voice1AfterDemodulation.wav



Voice2AfterDemodulation.wav

课程设计

课程设计题目1：傅里叶变换可以将复杂的时间信号转化到频域，然后用相对简单的频谱特性去分析原信号的动态特性。然而，傅里叶变换需要使用一个信号的所有时域信息，其对频谱的描绘是全局性的，使得我们无法看出特定频率所对应的时间信号发生的位置与持续时间。本题使用一种简单的时频分析工具，即短时傅里叶变换，探索音频信号的时频特性。短时傅里叶变换公式为：

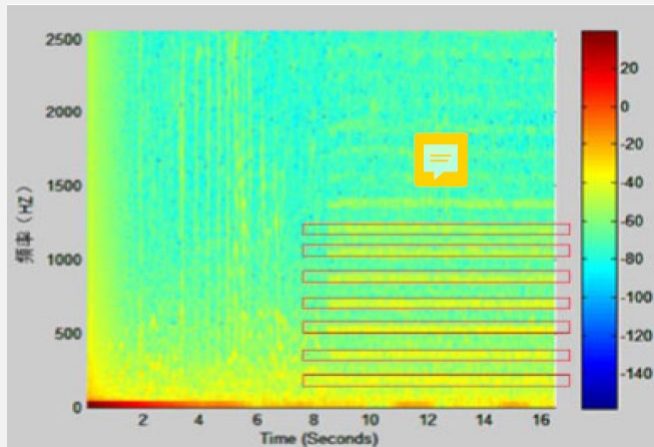
$$F_s(\omega) = \int_{-\infty}^{+\infty} f(\tau)h(\tau - t)e^{-j\omega\tau}d\tau$$

其中 $h(\tau)$ 为分析窗函数。由上式知，信号 $f(\tau)$ 在时间 t 处的短时傅里叶变换就是信号乘上一个以 t 为中心的窗函数 $h(\tau - t)$ 的傅里叶变换。
要求：

- 1、根据给定的两个信号数据，画出其傅里叶变换，对比两者的频谱特性。该信号采样率为4410Hz。
- 2、令为矩形窗函数，即

$$h(\tau) = \begin{cases} 1, & |\tau| \leq W \\ 0, & |\tau| > W \end{cases}$$

其中 W 为窗口大小。实现短时傅里叶变换函数，画出上述两个信号的时频图，探讨在不同窗口大小下的时频图规律。时频图格式如下，横坐标为时间，纵坐标为频率，颜色表示频率分量的幅值大小。



- 3、自选一段mp3 音乐，利用自己编写的短时傅里叶变换，做出该音乐的时频动画。动画效果参考<https://www.bilibili.com/video/av667556695/>

音频信号的短时傅里叶变换分析

Short-time Fourier Transform Analysis of Audio

1 背景介绍

傅里叶变换可以将复杂的时间信号转化到频域，然后用相对简单的频谱特性去分析原信号的动态特性。然而，傅里叶变换需要使用一个信号的所有时域信息（积分区域是 $(-\infty, +\infty)$ ），其对频谱的描绘是全局性的，使得我们无法看出特定频率所对应的时间信号发生的位置与持续时间。因此，对于上述问题需要用到一种简单的视频分析工具，即短时傅里叶变换。其公式可表示为：

$$F_s(w) = \int_{-\infty}^{\infty} f(\tau)h(\tau - t)e^{-jw\tau}d\tau \quad (1)$$

其中 $h(\tau)$ 为分析窗函数。由上式可知，信号 $f(\tau)$ 在时间 t 处的短时傅里叶变换就是信号乘上一个以 t 为中心的窗函数 $h(\tau - t)$ 后的傅里叶变换。

而原信号 $f(\tau)$ 的采样信号 $x[n]$ ，可写出其短时傅里叶变换公式为：

$$X[w] = \sum_{n=0}^{N-1} w[n]x[n]e^{-jw\omega}, 0 \leq \omega \leq 2\pi \quad (2)$$

其中 $w[n]$ 为新的窗函数。

在本次报告中采用 Matlab 软件进行分析，由于软件本身设计，尽可能以矩阵运算避免了循环代码的步骤，从而大幅度降低了程序的时间复杂度。

2 算法描述

2.1 原始数据频谱分析

对所给采样数据进行频谱分析用到最简单的傅里叶变换，代码详见附录 I。

所给采样数据被存储在 1×14000 的行向量 z 里面，定义 1×257 的行向量 Omega 用于存放频

率， 1×14000 的行向量 n 存储 0 到 13999 的 14000 个整数。通过式子

$$w = -jn' \cdot \text{Omega} \quad (3)$$

可以得到一个 14000×257 的矩阵，用于存储所有要用的 $n\omega$ 数据。再对 w 中每个位置的量取自然指数，即可得到一个 14000×257 的矩阵 e 。

因此矩阵 e 中第 i 行第 k 列的数据即代表着

$$n = i - 1; \omega = \frac{2\pi}{256} * (k - 1) \text{ 时的 } e^{-jn\omega}$$

从而由下式便可得到傅里叶采样数据的傅里叶变换：

$$\text{spectrum} = z \cdot e \quad (4)$$

2.2 加入窗函数后的时频分析

对所给采样数据进行频谱分析用最简单的傅里叶变换，代码详见附录 II。当我们引入的窗函数如下所示时，即意味着在选取点的前后 W 范围内的信号进行傅里叶变换。

$$w[n] = \begin{cases} 1, & |n| < W \\ 0, & |n| > W \end{cases} \quad (5)$$

我们将整个信号数据分为 100 段进行处理，分别选取窗口大小为 0.05s、0.1s、0.2s 和 0.4s 的窗函数进行分析讨论，信号首尾处不足一个窗口的信息用 0 补齐。

2.3 音频处理

对音频进行处理的代码如附录 III 所示。选取的音频的采样频率为 44100Hz，其数据量庞大，我们设置窗口大小为 0.1s，即单次测量 4410 个数据点。并且每过 0.1s 进行一次采样，重新以当前时间点前后提取 0.1s 的信号进行短时傅里叶分析，并将所得结果按照频率分为 8 个区域，从而能够更方便的进行展示。

将每次短时傅里叶变换得到的数据进行编码，导入到 MEGA2560 中，并用 8×8 的点阵进行展示。

3 实验结果

3.1 原始数据频谱分析

如将 1.(a)和图 1.(b)所示即为原所给信号 1 和 2 的时域图，其中采样频率为 4410Hz。

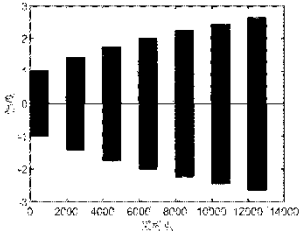


图 1 (a) 信号 1 的时域图

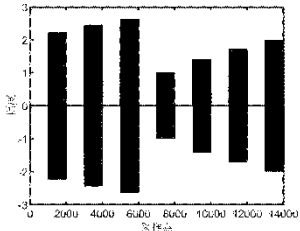


图 1 (b) 信号 2 的时域图

如图 2.(a)和图 2.(b)即分别为原始信号 1 和信号 2 的频谱分析结果。

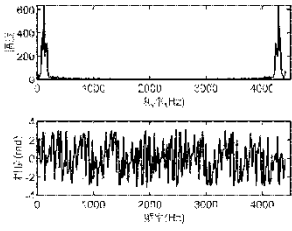


图 2 (a) 信号 1 的幅频特性和时域特性分析图

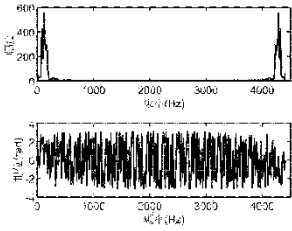


图 2 (b) 信号 2 的幅频特性和时域特性分析图

可以看到的是，信号 1 和信号 2 的幅频特性基本一致，在接近的频率附近出现幅度峰值。但是信号 2 的相频特性相对密集，临近频率的相位差异偏大。而另一方面，对于两个信号进行分析也可以说明，对于非周期信号单纯进行傅里叶变换的意义并不大，由此引入后短傅里叶变换，在部分时间内寻找这一段信号的频谱，并探索频谱随时间的变化。

3.2 加入窗函数后的时频分析

如图 3 所示为对信号 1 进行的时频分析，其中图 3.(a)、图 3.(b)、图 3.(c)和图 3.(d)分别代表了窗口大小为 0.05s、0.1s、0.2s 和 0.4s 的窗函数情况。

况。

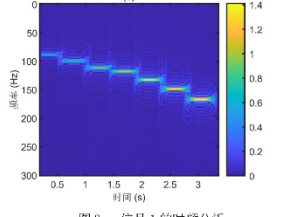
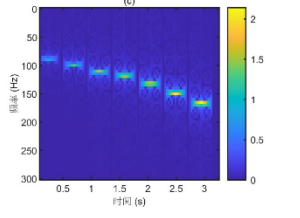
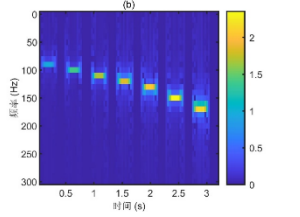
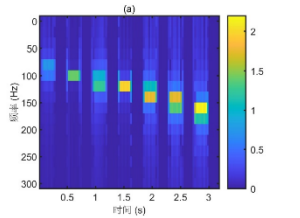


图 3 信号 1 的时频分析

如图 4 所示为对信号 2 进行的时频分析，其中图 4.(a)、图 4.(b)、图 4.(c)和图 4.(d)分别代表了窗口大小为 0.05s、0.1s、0.2s 和 0.4s 的窗函数情况。

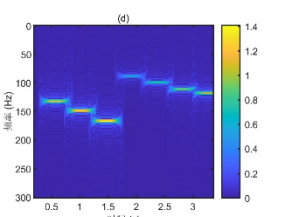
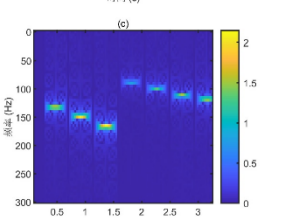
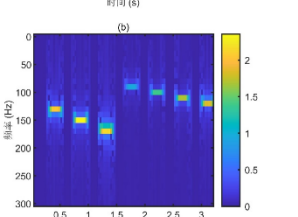
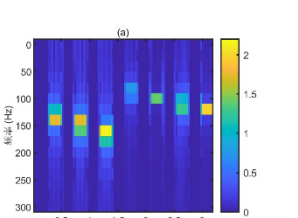


图 4 信号 2 的时频分析

课程设计题目1优秀课程设计论文展示：

3.3 音频处理

音频结果可见附带视频所示，点阵如图 5 所示。

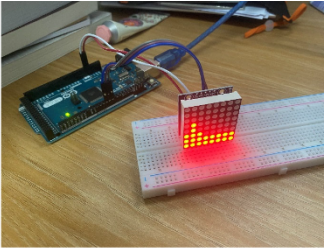


图 5 用于演示的点阵

附录I. 题目所给采样数据傅里叶变换源代码

```
1. clc; clear;
2. load('signal1.mat');
3. Omega = 0:pi/128:2*pi; % 1*NOspec
4. n = 0:1:length(z)-1; % 1*NOpoints
5. w = -1i*n'*Omega; %
6. e = exp(w);
7. spectrum = z * e;
8. subplot(2,1,1); plot(Omega, abs(spectrum)
, 'k')
9. subplot(2,1,2); plot(Omega, angle(spectru
m), 'k')
```

附录II. 采样数据时频分析代码

```
1. function Result=TFplot(data,N,dt)
2. %Fs 为每秒采样点数 4410
3. data=data';
4. Fs=4410;
5. % 音频长度
6. total_frame = size(data,1);
7. %采样间隔
8. delta=round(total_frame/N);
9. %傅里叶变换窗口
10. half_space = floor(dt*Fs/2);
```

```
11. %时间序列
12. T = [1:delta:total_frame];
13. T=T+half_space;
14. LT = length(T);
15. %音频前后补充 0
16. data = data';
17. data = [zeros(1,half_space),data,zeros(1,
half_space)]';
18. L = 2*half_space+1;
19. Result=zeros(1,floor(L/2)+1);
20.
21. for Current=T
22. left = Current - half_space;
23. right = Current + half_space;
24. X = data(left:right,:);
25. % 离散傅里叶 Dft
26. Y = Dft(X,L);
27. Y = Y';
28. Result=[Result;Y];
29. end
30.
31. Result = Result';
32. x=T/Fs;
33. ymax = floor(300*L/Fs);
34. y=Fs*(0:ymax)/L;
35. Result1 = Result(:,2:end);
36. Result2 = Result(1:ymax+1,:);
37. %subplot(2,1,1);
38. imagesc(x,y,Result2);
39. colorbar;
40. xlabel('时间 (s)');
41. ylabel('频率 (Hz)');
```

附录III. 音频短时傅里叶变换代码

```
1. %全局句柄
2. global bar_handler;
3. global music;
4. %Fs 为每秒采样点数 44100
5. [data, Fs] = audioread('震撼上场背景音乐.mp3',[1,44100*40]);
6. % 音频长度
```

```
7. total_frame = size(data,1);
8. % 更新音乐播放器
9. music = audioplayer(data, Fs);
10. % 创建一个定时器。
11. t = timer('Period', 0.1, 'TimerFcn', {@ti
mer_callback,data,Fs,total_frame},'Execu
tionMode','fixedSpacing');
12. % 启动定时器，每隔 0.1 秒调用一帧动画
13. start(t);
14. % 进行播放
15. play(music);
16.
17.
18.
19.
20. %定时器回调函数
21. function timer_callback(object, eventdata
,data,Fs,total_frame)%
22. global music;
23. global bar_handler;
24. Current = music.CurrentSample;
25. %确定取样范围
26. half_space = floor(0.1*Fs/2);
27. left = Current - half_space;
28. if left<1
29. left = 1;
30. end
31. right = Current + half_space;
32. if right>total_frame
33. right = total_frame;
34. end
```

```
35. X = data(left:right,:);
36. % 将音频左右声道合并
37. X = sum(X, 2);
38. L = length(X);
39.
40. % 离散傅里叶 Dft
41. Y = Dft(X,L);
42. freq2 = Fs*(0:floor(L/2))/L;
43. P2 = Y(freq2>20&freq2<2000);
44. freq2 = freq2(freq2>20&freq2<2000);
45. % 分为 32 份
46. nbins = 31;
47. % 每一份的宽度
48. window = floor(length(freq2)/nbins);
49.
50. % 平滑数据
51. P3 = smooth(P2, window);
52. P4 = P3(1>window:end);
53. freq3 = freq2(1>window:end);
54. P4 = round(P4*100);
55. %更新句柄属性
56. set(bar_handler, 'XData',freq3, 'YDa
ta',P4);
57. fid=fopen('data3.txt', 'a');
58. fprintf(fid,"%d %d %d %d %d %d %d
",P4(1),P4(2),P4(3),P4(4),P4(5),P4(6),P4(
7),P4(8));
59. fprintf(fid, '\n');
60. fclose(fid);
61. end
```

MATLAB MCP + 腾讯WorkBuddy的智能体尝试

示例3：用MCP + WorkBuddy做以上课程设计

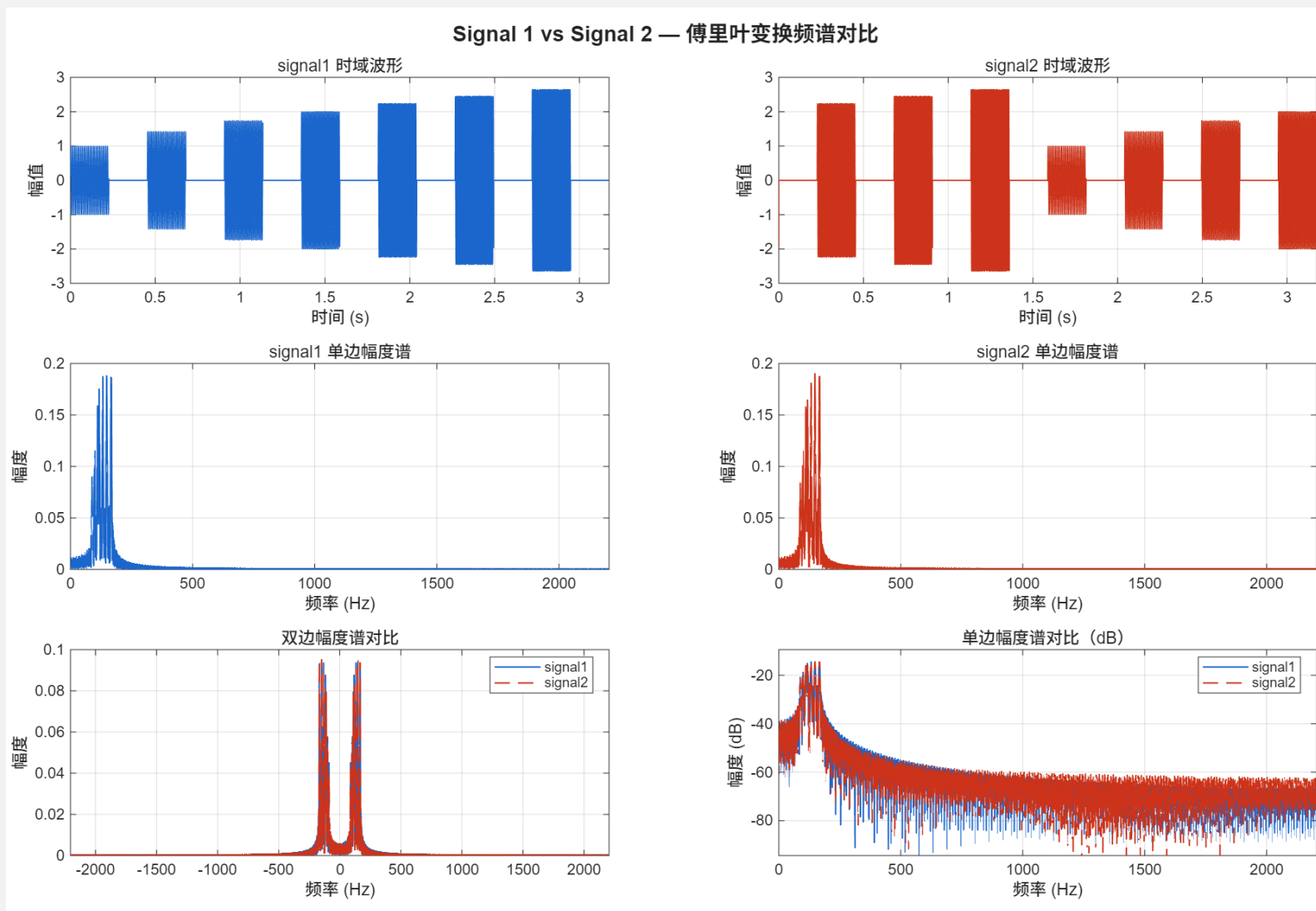
提示词：在D:/matlab/signals and systems/course design文件夹中，有两个信号文件，为signal1.mat和signal2.mat，两个信号的采样率都是4410Hz。（1）请使用MATLAB画出其傅里叶变换，并对比两者的频谱。（2）请使用矩形窗函数，其中 W 为窗口大小，画出上述两个信号的短时傅里叶变换的时间-频谱图，探讨在不同窗口大小下的时频图规律。时频图格式如下：横坐标为时间，纵坐标为频率，颜色表示频率分量的幅值大小。（3）D:/matlab/signals and systems/course design文件夹中有一个music.mp3音乐文件。请利用自己编写的短时傅里叶变换，做出该音乐的时频动画。动画效果参考 <https://www.bilibili.com/video/av667556695/>



Music.mp3

MATLAB MCP + 腾讯WorkBuddy的智能体尝试

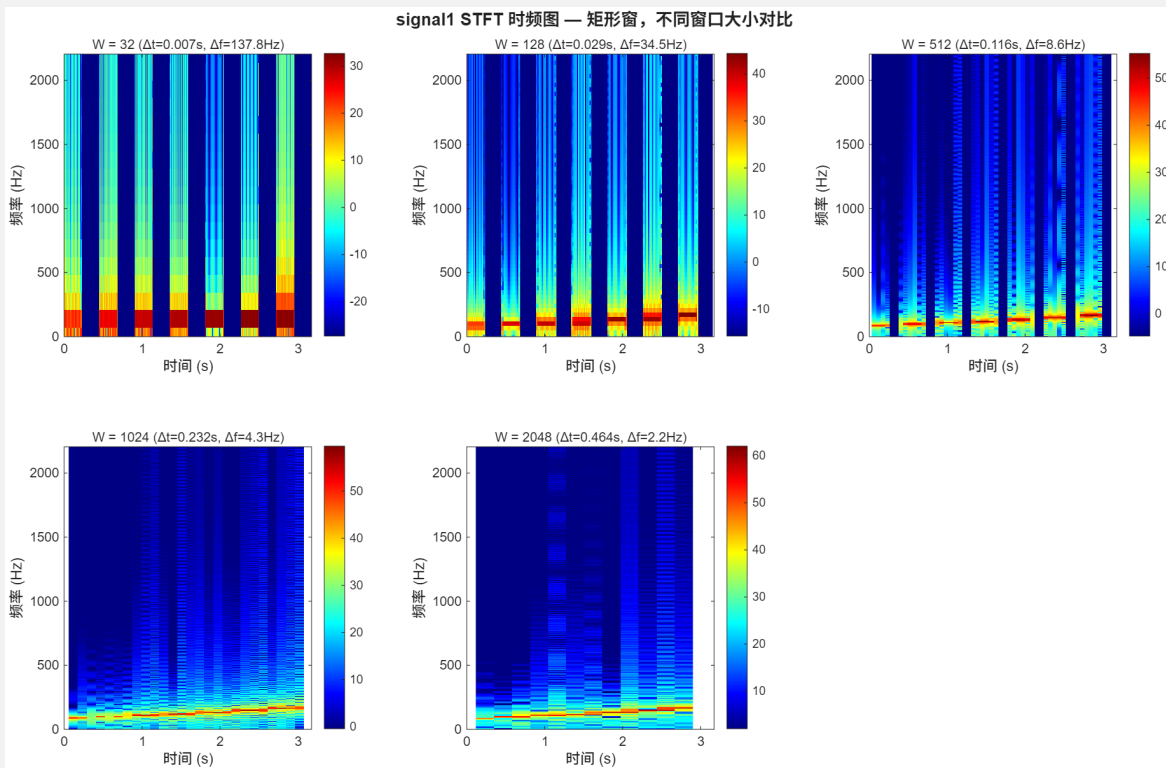
示例3：用MCP + WorkBuddy做以上课程设计



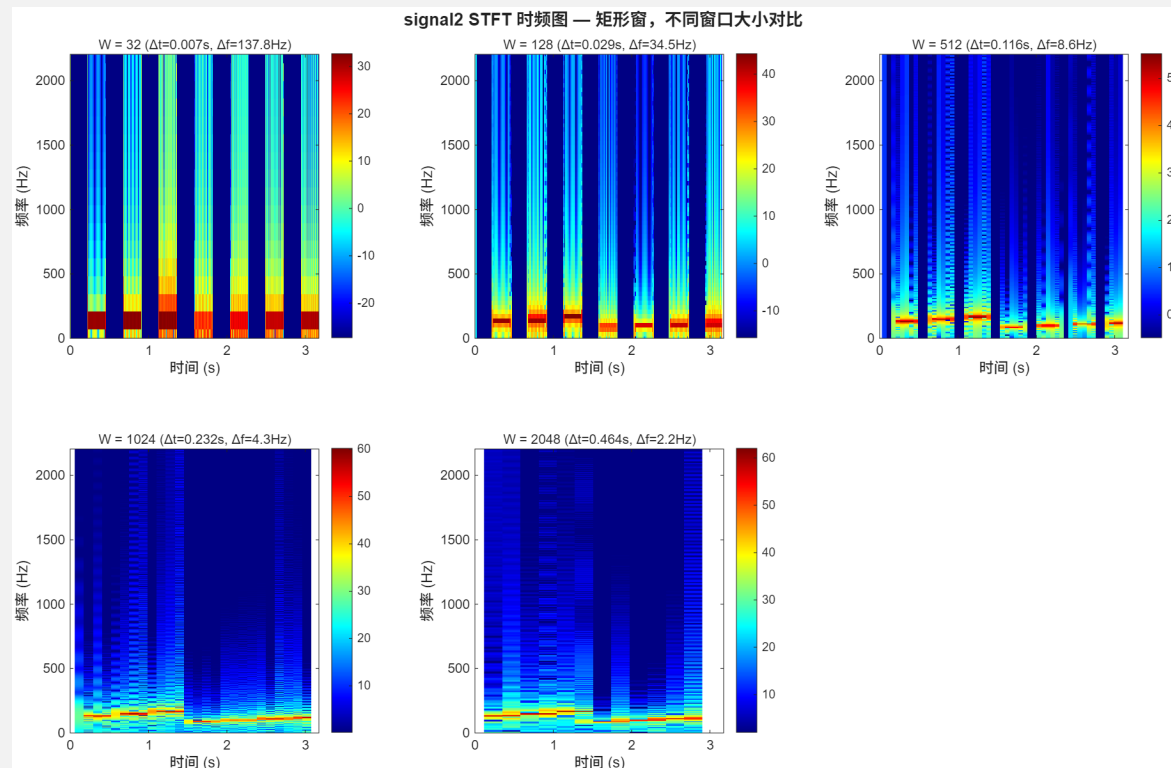
(1) 两个信号的频谱对比

MATLAB MCP + 腾讯WorkBuddy的智能体尝试

示例3：用MCP + WorkBuddy做以上课程设计



信号1的短时傅里叶变换



信号2的短时傅里叶变换

(2) 两个信号的在不同窗口长度下的短时傅里叶变换

MATLAB MCP + 腾讯WorkBuddy的智能体尝试

示例3：用MCP + WorkBuddy做以上课程设计



(3) 利用自己编写的短时傅里叶变换，做出该音乐的时频动画

MATLAB MCP + 腾讯WorkBuddy的智能体尝试

示例3：用MCP + WorkBuddy做以上课程设计

提示词：对于该音乐的时域动画，能否在输出的动画视频中也播放music.mp3相应的声音，形成“动画+声音”的多媒体形式并输出视频文件。

+

在文件夹D:/matlab/signals and systems中有一个segment2.wav文...

对于该音乐的时域动画，能否在输出的动画视频中也播放music.mp3相应的声音，形成“动画+声音”的多媒体形式并输出视频文件。

WorkBuddy

>

9 个工具调用，9 条过程消息

完成！音频合并已实现

已完成的工作

① FFmpeg 安装

已为这台电脑安装了 FFmpeg 8.1.1（路径已写死到脚本中，无需任何配置）：

C:\Users\Lenovo\AppData\Local\ffmpeg\ffmpeg-8.1.1-essentials_build\bin\ffmpeg.exe

② 验证 pipeline 成功

15 秒预览视频 + 音乐音频 → 合并成功

文件	大小
无声视频	14.1 MB
含音频视频	14.3 MB（↑ AAC 音频轨道）

（3）利用自己编写的短时傅里叶变换，做出该音乐的时频动画

MATLAB MCP + 腾讯WorkBuddy的智能体尝试

示例3：用MCP + WorkBuddy做以上课程设计

提示词：对于该音乐的时域动画，能否在输出的动画视频中也播放music.mp3相应的声音，形成“动画+声音”的多媒体形式并输出视频文件。



(3) 利用自己编写的短时傅里叶变换，做出该音乐的时频动画

在AI时代的实践类教学方式探讨

- (1) 你是否支持学生使用AI完成作业？
- (2) 你是否会在课堂上讲授如何利用AI完成作业？
- (3) 你是否用AI出题给学生做？
- (4) 你是否会倾向于出更难题目，以至于目前的AI水平难以圆满完成？
- (5) 你会用AI去评价学生提交的程序和实验报告吗？
- (6) 你是否会给AI味道不那么多的程序和实验报告更高的分数？
- (7) 作为教师，我们相比AI的优势是什么？
- (8) 在AI时代，我们培养的学生应该具备怎样的核心竞争力？



感谢专家
请多指导

勤学 / 修德 / 明辨 / 笃实